《NoSQL 数据库技术与应用》期末项目

智慧图书馆数字资源平台

——在三台 CentOS 虚拟机上用 Xshell 部署 MongoDB 单机、副本集与分片集群

姓　　名：郭雨昊

学　　号：202414870312

班　　级：24 数科 3 班

专　　业：数据科学与大数据技术

完成方式：单人独立完成

完成日期：2026 年 11 月 10 日

运行环境：CentOS Stream 10 (Coughlan) / kernel 6.12.0-269.el10.x86_64 / MongoDB 8.0.34

节点：nosql01 192.168.121.134、nosql02 192.168.121.135、nosql03 192.168.121.136

数据规模：图书 50 种 / 读者 30 人 / 借阅记录 3000 条 / GridFS 文件 6 个

实例端口：27017 单机、27018 副本集、27019 配置服务器、27020-27021 分片、27022 mongos

一、项目概述

1.1 任务与做法

课程任务书要求围绕“智慧图书馆数字资源平台”完成 NoSQL 数据库的设计、部署与验证。老师给的思路是：用 VMware 起三台 CentOS 虚拟机，在虚拟机里装 MongoDB，再用 Xshell 通过 SSH 连进去做实验。我的做法是按这个思路来的：三台机器分别叫 nosql01、nosql02、nosql03，连成一个小网络，在上面依次搭了单机实例（27017）、三节点副本集 rs_lib（27018）和分片集群（配置服务器 cfgrs、两个分片副本集 shard1rs 和 shard2rs、路由 mongos 27022），三个阶段都导入同一批数据，各自验证一遍。命令都在 Xshell 的 SSH 会话里敲，数据都放在虚拟机的 /data/lib 目录下。

有一点和最初想的不一样：电脑上现成的三台克隆机装的是 CentOS Stream 10（Coughlan），不是 CentOS 7.9。本来打算重新下 ISO 装 7.9，后来觉得镜像三个多 G，下载加安装太费时间，就直接用这三台克隆机做了。所以报告里的版本信息都是 CentOS Stream 10、内核 6.12.0-269.el10，配的是机器上已经装好的 MongoDB 8.0.34（mongosh 2.14.0、database-tools 100.20.0）。

网络这块最后用的是 VMware NAT 模式（192.168.121.0/24，网关 192.168.121.2）。按老师的思路先试了桥接，结果虚拟机拿不到 IPv4，宿主机也 ping 不通，查下来是本机只有一块无线网卡、VMnet0 没绑上物理网卡，只好退回 NAT，把原来的静态 IP 改回来。试桥接的经过和失败原因写在第九章。

1.2 交付物

表 1 交付物清单

类别	内容	位置 / 说明
技术文档	本报告（Word + PDF 各一份）	24 数科 3 班-202414870312-郭雨昊.docx / .pdf
数据库脚本	建索引、统计查询、账号权限、GridFS、副本集、分片、验收（12 个 mongosh 与 shell 脚本）	脚本/ 目录
运行日志	每个阶段一份完整命令与输出	日志/（22 份阶段日志、5 份网络排查记录）
过程截图	虚拟机控制台、Xshell 会话、终端输出	截图/
数据导出	books / readers / borrow_logs / 分类统计 JSON	/data/lib/out/（19,396 / 7,815 / 647,960 / 742 B）
代码仓库	https://github.com/winpay585/library-mongodb-lab	main 分支

文档按“24 数科 3 班-202414870312-郭雨昊”命名，交给课代表汇总。

二、实验环境与集群拓扑

2.1 三台虚拟机的规格

表 2 三台虚拟机规格与实例分布

节点	IP（NAT）	配置	操作系统 / 内核	承担的实例
nosql01	192.168.121.134	4 GB 内存 / 2 vCPU / 3.37 GB 磁盘	CentOS Stream 10 (Coughlan)	单机 27017、副本集 27018、配置 27019、分片 shard1 27020、mongos 27022
nosql02	192.168.121.135	4 GB 内存 / 2 vCPU（链接克隆）	CentOS Stream 10 (Coughlan)	副本集 27018、配置 27019、分片 shard1 27020、分片 shard2 27021
nosql03	192.168.121.136	4 GB 内存 / 2 vCPU（链接克隆）	CentOS Stream 10 (Coughlan)	副本集 27018、配置 27019、分片 shard2 27021

三台虚拟机由 VMware Workstation 16.1.2 管理，02、03 是 01 的链接克隆（共用 NOSQL01-cl1.vmdk 基础镜像），因此三台的软件环境天然一致：MongoDB 8.0.34 已预装，另带一份上一门课遗留的 /etc/mongod.conf（端口 27016、bindIp 127.0.0.1、dbPath /var/lib/mongo，约 204 MB 旧数据）。收尾的时候把这个遗留实例删掉了，过程见 9.3 节。

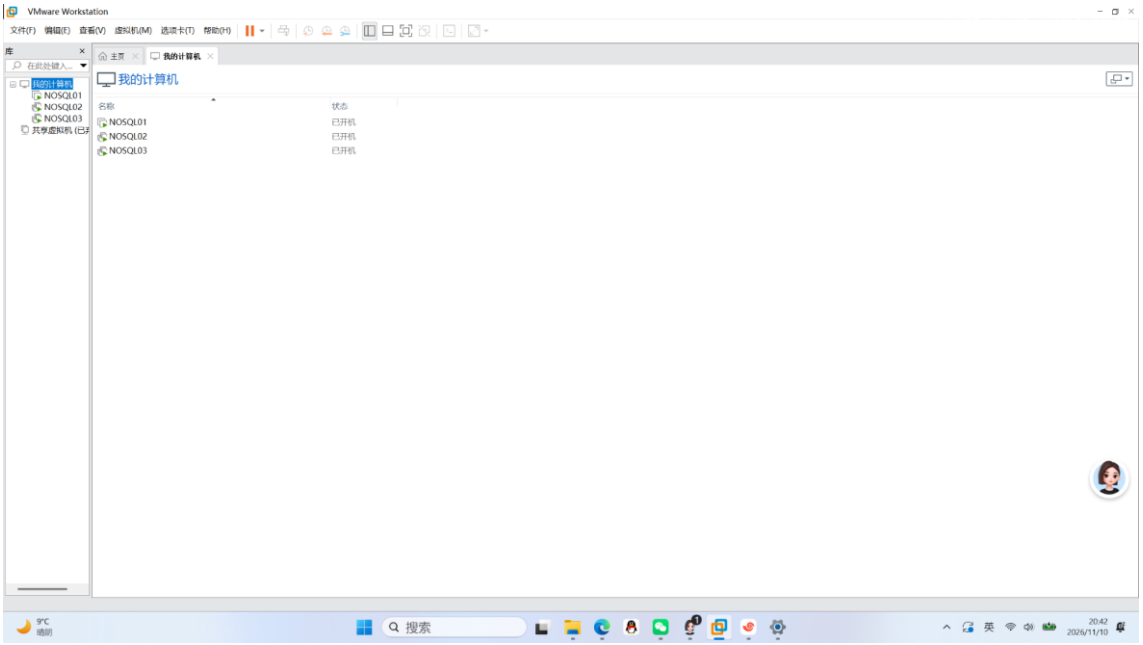


图 1　VMware Workstation 中三台虚拟机均处于开机状态

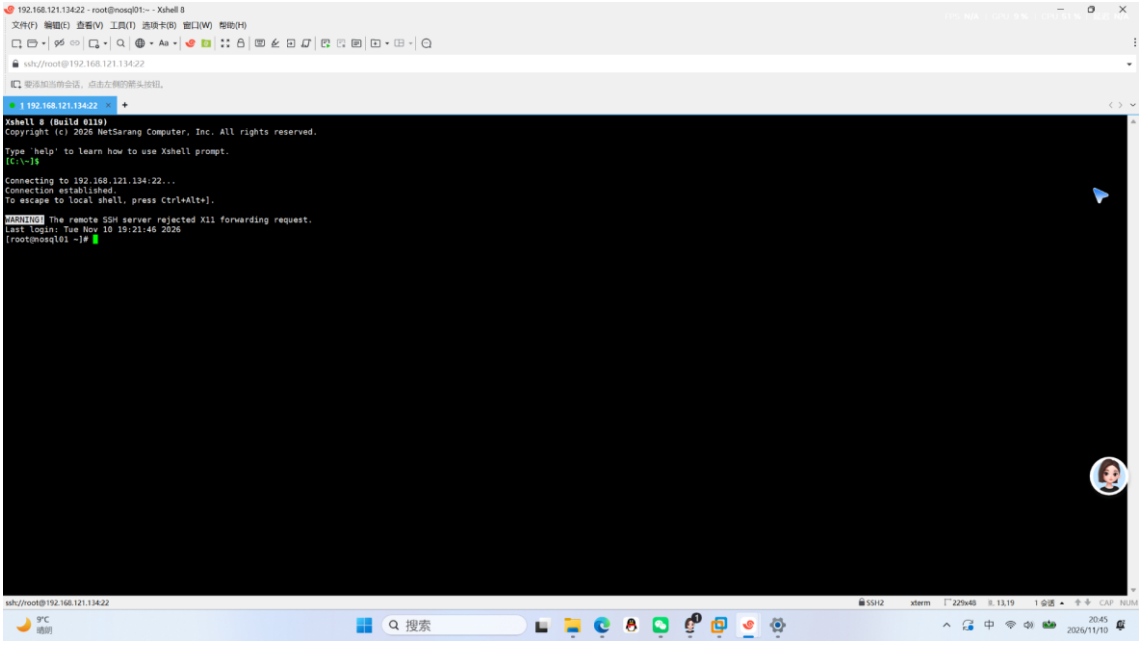


图 2　Xshell 8 通过 SSH 登录 nosql01（ssh://root@192.168.121.134:22）

2.2 端口与目录规划

三台机器共用一套目录规范：项目根为 /data/lib，下设 log（日志）、conf（配置）、key（密钥）、tmp（临时与数据）、以及每个实例独立的数据目录（standalone、rs1-rs3、cfg1-cfg3、shard1、shard2）和 out（导出）。这样做的好处是：一个实例出问题只影响它自己的数据目录，删除重建时 rm -rf 的目标是明确的，不会误伤别的实例。

表 3 实例与端口分配

实例	端口	节点	数据目录	角色
standalone	27017	nosql01	/data/lib/standalone	单机实例（开启认证）
rs_lib	27018	三台各一	/data/lib/rs1~rs3	副本集，nosql01 priority 2
cfgrs	27019	三台各一	/data/lib/cfg1~cfg3	配置服务器副本集（configsvr）
shard1rs	27020	nosql01 + nosql02	/data/lib/shard1	分片副本集，nosql01 priority 2
shard2rs	27021	nosql03 + nosql02	/data/lib/shard2	分片副本集，nosql03 priority 2
mongos	27022	nosql01	（无独立数据目录）	路由节点，挂 cfgrs

配置文件统一放在 /data/lib/conf 下，命名与实例一一对应（standalone-auth.conf、rs1.conf、cfg1.conf 、 shard1.conf 、 mongos.conf 等 ） ， 日 志 写 在 /data/lib/log 下 。 密 钥 文 件 /data/lib/key/mongo.key 是三台一致的 64 字符十六进制串，权限 400，副本集与分片集群的所有成员共用它——这是内部认证（--keyFile）的前提。

三、第一阶段：单机实例（27017）

单机这一步要做的事情有五件：检索、统计、索引、文件存储、权限，都要在机器上真的跑出结果来，同时把数据和账号准备好，后面两个阶段接着用。

3.1 实例配置与数据导入

standalone.conf 的关键项：port 27017、bindIp 0.0.0.0（让另外两台和宿主机都能连）、dbPath /data/lib/standalone、logPath /data/lib/log/standalone.log、fork true、wiredTiger cacheSizeGB 0.8（虚拟机只有 4 GB 内存，必须限住缓存，否则三台同时起多实例会 OOM）。

数据用 mongoimport --jsonArray --drop 导入三个集合：books 50 条、readers 30 条、borrow_logs 3000 条。导出体积分别是 19,396 B、7,815 B、647,960 B——借阅记录一个集合就占了总量的 96%，这也正是后面要给它单独设计索引与分片键的原因。

```
[root@nosql01 ~]# mongoimport --host 127.0.0.1 --port 27017 -d library -c books \
                  --file /data/lib/tmp/data/books.txt --jsonArray --drop
2026-11-10T19:02:41  connected to: mongodb://127.0.0.1:27017
2026-11-10T19:02:41  dropping: library.books
2026-11-10T19:02:41  50 document(s) imported successfully. 0 document(s) failed to import.
[root@nosql01 ~]# mongosh --port 27017 --quiet --eval 'const d=db.getSiblingDB("library");
  print(d.books.countDocuments(), d.readers.countDocuments(), d.borrow_logs.countDocuments())'
50 30 3000
```

3.2 索引设计

建索引是看查询怎么写就怎么建。books 上有 5 个索引（含 _id_）：uk_isbn 保证 ISBN 唯一，idx_category 支持按分类筛选，idx_title_text 是文本索引，供书名模糊检索，idx_cat_isbn 是“分类 + ISBN”复合索引，让“某分类下按 ISBN 排序”的查询只走一次索引。readers 4 个索引，borrow_logs 6 个索引，其中 idx_card_hashed 是哈希索引，既服务于“按借阅证查记录”，也是后面分片集群的分片键前缀索引。

```
[root@nosql01 ~]# mongosh --port 27017 -u platform_admin -p Admin2026pwd \
                  --authenticationDatabase admin --quiet --eval '
  const d = db.getSiblingDB("library");
  ["books","readers","borrow_logs"].forEach(c => print(c + ": " +
d[c].getIndexes().map(i=>i.name).join(", ")))'
books: _id_, uk_isbn, idx_category, idx_title_text, idx_cat_isbn
readers: _id_, uk_card_no, idx_college, idx_college_card
borrow_logs: _id_, idx_card_hashed, idx_book_isbn, idx_date_desc, idx_isbn_date, idx_status
```

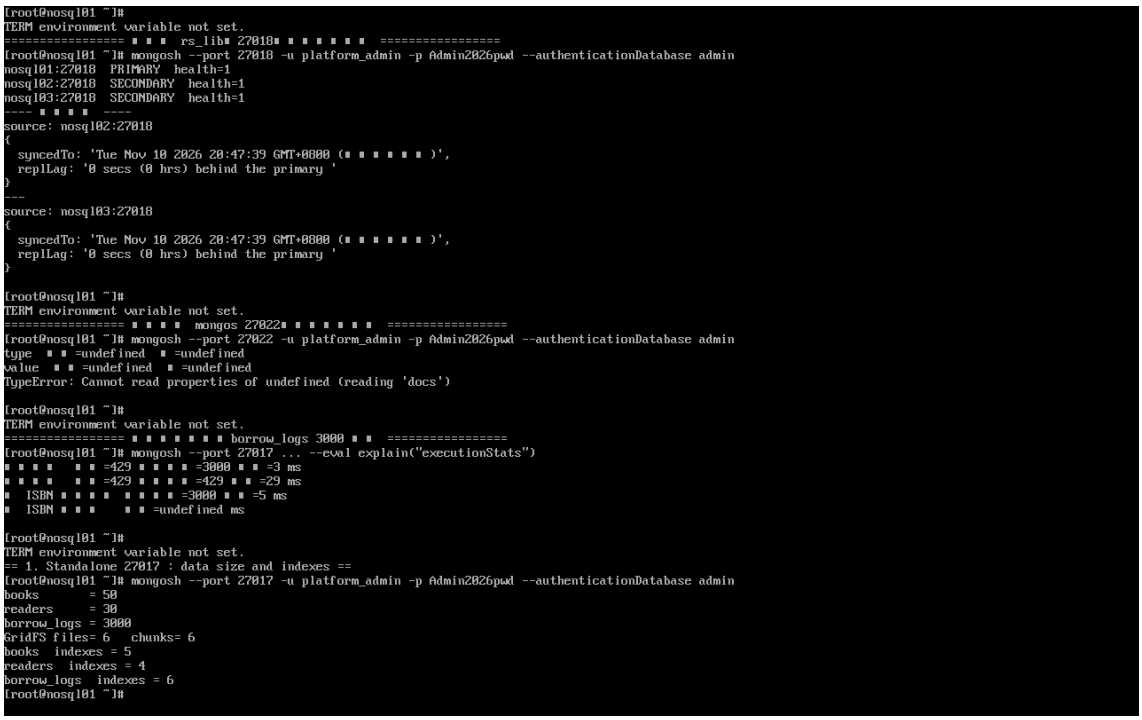


图 3 单机实例：三集合数据量、GridFS 文件数与索引数量（控制台实测输出）

3.3 五类统计（聚合管道）

统计全部用聚合管道实现，结果与 data 目录里的源数据逐条核对过。

表 4 五类统计实测结果

统计项	统计方式	实测结果
Q1 分类统计	按 category 分组：册数、平均价、总借阅量	计算机 15 本 / 均价 89.53 / 借阅 2049；文学 12 / 47.02 / 1744；经济管理 7 / 90.86 / 617；数学 6 / 40.67 / 1086；历史 5 / 95.60 / 638；外语 5 / 59.00 / 653
Q2 学院分布	按 college 分组取前 5	经济管理学院 5、计算机学院 5、外国语学院 4、管理学院 4、数学与统计学院 4
Q3 月度借阅	按 borrow_date 的年月分组	36 个月，2023-01 为 86 条，2025-12 为 76 条，合计 3000 条
Q4 借阅 Top10	按 book_isbn 分组计数降序取 10	10 本书并列各 60 次（小王子、红楼梦、全球通史、国富论、局外人、费马大定理、边城、Python 数据分析、算法图解、高等数学（上册））
Q5 状态统计	按 status 分组：条数、平均借阅天数	已归还 2142 条（均 44.6 天）、逾期 429 条（44.5 天）、借出 429 条（44.2 天）

这里踩过一个坑：首轮把 $round 直接写在 $group 的累加器位置，报 “The $round accumulator is a unary operator”。$round 是单目运算符，不能当累加器，正确写法是先用 $avg 求平均，再在 $project 阶段套 $round。改完之后五类统计才全部出数。

3.4 GridFS 电子书存取

三个 PDF 电子书加三张封面图通过 mongofiles put 写入 fs.files / fs.chunks，再落盘下载做 MD5 比对，六个文件全部一致，说明存进去的字节和拿出来的是同一份。

表 5 GridFS 文件完整性校验

文件	源 MD5	入库后下载 MD5	结论
ebook_1.pdf	f818f1a754c2fabfdddf96f86294b9b4	f818f1a754c2fabfdddf96f86294b9b4	一致
ebook_2.pdf	c86c3991f2b1797d93839034f1a06ba9	c86c3991f2b1797d93839034f1a06ba9	一致
ebook_3.pdf	3536df7adad16f5e9894d6e37f55ebd0	3536df7adad16f5e9894d6e37f55ebd0	一致
cover_1.png	7e44bcff0206b927fbb63cc5340861cd	7e44bcff0206b927fbb63cc5340861cd	一致

文件	源 MD5	入库后下载 MD5	结论
cover_2.png	eb16d7458ed9d95193e1d2f7bfeddec6	eb16d7458ed9d95193e1d2f7bfeddec6	一致
cover_3.png	8e4d140cd5674f7da8d86f6efbfe88fa	8e4d140cd5674f7da8d86f6efbfe88fa	一致

这一步也遇到两个问题。一是 mongofiles 用绝对路径上传时，会把整条路径当成文件名存进 fs.files，下载时按文件名找就报 not found；必须先 cd 进目录、用相对文件名。二是开启认证后再传，命令不带 -u/-p 会全部返回 (Unauthorized)，必须显式带账号并指定 --authenticationDatabase library。两条都改完后六个文件才一次全部通过。

3.5 权限体系与认证

单机阶段最后开启了 security.authorization: enabled（配置存为 standalone-auth.conf，当前运行的正是它），并按三层划分账号：平台管理员、业务库管理员、业务读写、业务只读。

表 6 账号与权限划分

账号	认证库	角色	说明
platform_admin	admin	root	平台级管理，用于建库、建账号、查看状态
lib_admin	library	dbOwner	业务库所有者，可建集合与索引
lib_editor	library	自定义 library_editor	只有 find/insert/update/remove 四个动作
lib_reader	library	read	只读，用于查询端

权限验证里发现一个真实问题：lib_editor 最初给的是内置 readWrite 角色，结果它能直接 drop 集合——readWrite 包含 dropCollection，这与“编辑只能改数据不能删集合”的设计意图不符。于是新建自定义角色 library_editor（privileges 只给 find、insert、update、remove），用 updateUser 挂到 lib_editor 上，复测删除集合返回 Unauthorized，权限边界才真正立住。这个细节说明：内置角色的粒度是按“对象能做什么”，而不是按“业务上允许做什么”，业务边界必须自己定义角色。

四、第二阶段：副本集 rs_lib（27018）

4.1 从单机到三节点的改造

副本集不是把同一份配置复制三份，而是在每台机器上起一个独立 mongod，配置里加 replSetName: rs_lib 与 keyFile，然后在一台上执行 rs.initiate，让三台互相认出对方。三台的 /etc/hosts 已预先写好三条映射，所以副本集成员名直接用主机名（nosql01:27018 这类），比用 IP 更容易读，也避免 NAT 环境下 IP 变更导致配置失效。

```
[root@nosql01 ~]# mongosh --port 27018 -u platform_admin -p Admin2026pwd \
                --authenticationDatabase admin --quiet --eval '
  rs.status().members.forEach(m => print(m.name + "  " + m.stateStr + "  health=" + m.health))'
nosql01:27018  PRIMARY    health=1
nosql02:27018  SECONDARY  health=1
nosql03:27018  SECONDARY  health=1
```

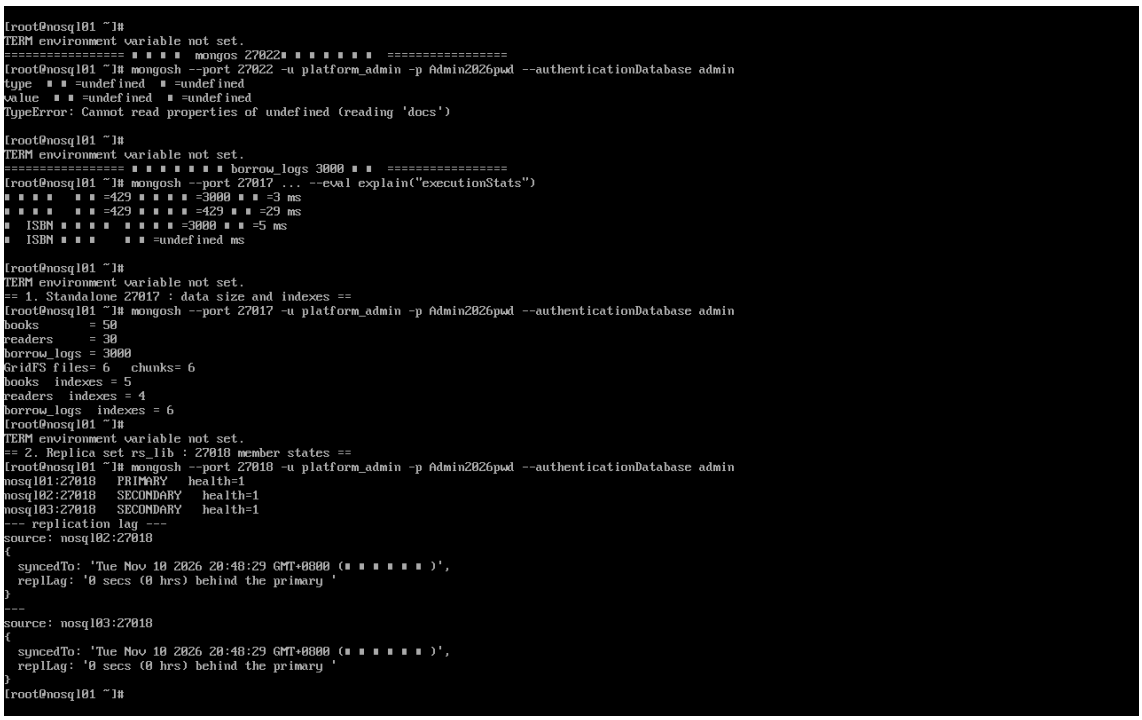


图 4 副本集三节点状态与复制延迟（replLag 0 秒）

建集过程并不顺利，主要卡在两处。第一处是 27018 端口被更早遗留的进程占着，运行中直接 rm -rf 数据目录，触发 WiredTiger 的 WT_PANIC（exitCode 48）——这提醒一件事：删数据目录前必须先确认进程已退出，否则引擎打开的是“半删”的目录。第二处是清理旧实例时用 pkill -f 'rs1.conf'，模式把执行这条命令的进程自己也匹配上，终端当场断开、命令返回 exit -1。后来改成先用 ss -lntp 解析出占用端口的 pid，再对具体的 pid 执行 kill -9，三台清干净、27017-27024 全部空闲后重来。

第二类坑在账号上。mongosh --port 27018 默认连的是 localhost，解析到 IPv6 的 ::1，不享受“本机连接免认证”的例外，于是 createUser 报 requires authentication；改用 mongodb://127.0.0.1:27018/?directConnection=true 才建出 platform_admin。而且这个例外只允许在 admin 库建第一个用户，library 库的账号仍然建不出来，最后是用 platform_admin 去建三个业务账号。也就是说，开了认证的副本集要建账号，要么走 localhost 例外（只在 admin 库第一个用户管用），要么先有个管理员账号。

4.2 故障转移实测

演练方式：在主节点 nosql01 上 kill 掉 mongod（pid 6209），观察副本集重新选主的时间与新主，再让原主重新加入、确认它按 priority 2 回主且数据追平。

表 7 故障转移演练记录

环节	观察结果
主节点被杀	nosql01:27018 进程终止，副本集失去主节点
重新选主	13.57 秒后 nosql03:27018 当选新主（SECONDARY → PRIMARY）
故障期写入	在新主上写入 1 条探针记录，books 变为 51
原主状态	rs.status() 中 nosql01 显示 not reachable，health 0
重启原主	nosql01 重新加入并按 priority 2 回主，books 仍为 51，说明故障期写入已同步追平
复制延迟	rs.printSecondaryReplicationInfo() 两台均 replLag 0 secs
收尾	删除探针记录，数据回到 50 / 30 / 3000

13.57 秒大致是两个部分：选举超时（默认 electionTimeoutMillis 10 秒），加上候选者拉票、新主切换状态的耗时。也就是说，默认参数下三节点副本集的最短不可写窗口在 10 秒以上，对“图书馆借还”这类业务可以接受，但对强实时系统就需要调小 electionTimeout 或改用更快的仲裁策略。另外，杀主而不杀整台机器，与整机断电的效果不同：前者只是进程消失，后者还可能带走未刷盘的数据，所以生产上要同时验证这两种故障。

五、第三阶段：分片集群

5.1 集群骨架

分片集群由三类角色组成：配置服务器（存集群元数据，本身必须是副本集）、分片（存实际数据，每个分片也是副本集）、路由（mongos，对外提供统一入口）。本项目的骨架是：cfgrs 三节点（27019）、shard1rs 两节点（nosql01 + nosql02，27020）、shard2rs 两节点（nosql03 + nosql02，27021）、mongos 单节点跑在 nosql01:27022。

```
[root@nosql01 ~]# mongosh --port 27022 -u platform_admin -p Admin2026pwd \
                  --authenticationDatabase admin --quiet --eval '
  db.adminCommand({listShards:1}).shards.forEach(s => print(s._id, s.host))'
shard1 shard1rs/nosql01:27020,nosql02:27020
shard2 shard2rs/nosql03:27021,nosql02:27021
```

注意 nosql02 同时是 shard1rs 和 shard2rs 的成员，两个副本集各占一个端口（27020、27021）、各自的 keyFile 与数据目录。这是在三台机器的约束下让两个分片都具备副本冗余的最小做法；代价是 nosql02 承担了两份数据，真实生产应当让每个分片成员尽量分散到不同物理机。

5.2 分片键的选择与论证

表 8 分片键设计

集合	分片键	类型	选择理由
borrow_logs	{reader_card: "hashed"}	哈希	3000 条记录、按借阅证查询最频繁；哈希能把写请求均匀打散，避免按证号范围聚集
books	{isbn: "hashed"}	哈希	50 本书按 ISBN 哈希分布，避免自增式热点；查询多按 ISBN 等值命中
readers	不分片	—	仅 30 条，留在主分片即可，分片反而增加路由开销

这里踩了两个坑。第一个是顺序：先 mongoimport --drop 导入数据、再 shardCollection，会失败——因为 --drop 会把集合连同“已分片”的元数据一起清掉，集合退回未分片状态，split 一律报 NamespaceNotSharded。正确顺序是“导入 → 建索引 → 分片”。其二是索引前置：哈希分片键必须先建 {isbn:"hashed"} 这类前缀索引，否则 shardCollection 直接报 InvalidOptions，具体提示是“Please create an index that starts with the proposed shard key”。这条提示是把 09_分片集群搭建.js 跑起来才看到的，最后就是照着这句报错加索引改的。

5.3 数据分布与手动迁移

集合分片成功后先做了一次 3 分钟的均衡观察：balancerStatus 显示 mode full、numBalancerRounds 74、inBalancerRound false，mongos.log 里没有任何迁移报错，但两块数据仍全在 shard1。原因是数据量只有 582 KiB，每个分片只有一个块，均衡器判定“已平衡”不会动手——均衡器迁移的前提是块数多于分片数。

为了演示真实分布，改用 moveChunk 手动迁移：borrow_logs 4 块、books 1 块迁到 shard2，全部返回 ok。迁移后的分布如下（重启集群后 MongoDB 会把小片合并成大块，因此块数从 8/2 变成每分片 1 块，文档比例不变）。

表 9 分片后的真实数据分布

集合	shard1	shard2	合计
borrow_logs	1300 条 / 252 KiB（43.34%）	1700 条 / 329 KiB（56.65%）	3000 条 / 582 KiB
books	21 本 / 8 KiB（42.35%）	29 本 / 11 KiB（57.64%）	50 本 / 19 KiB

5.4 路由验证

分片集群最重要的一条结论是：查询是否带分片键，决定了它是“单分片命中”还是“广播到全部分片”。用 explain 的 winningPlan.shards 可以直接看到实际访问了哪些分片。

表 10 查询路由实测

查询	是否带分片键	命中的分片
{reader_card: "R20240001"}	是	shard1
{isbn: "9787115428028"}	是	shard1
{status: "逾期"}	否	shard1, shard2

这也解释了为什么业务查询要尽量把分片键带上：不带分片键的查询要打到所有分片再合并结果，分片越多、代价越大，本质上是把并行度换成了协调开销。

六、索引与查询性能验证

索引到底有没有用，看 executionStats 里的扫描量最直接。下面几组对比都在单机 27017 上跑，集合固定为 borrow_logs 3000 条、books 50 条，唯一变量是走不走索引。

表 11 索引效果对比

查询场景	方式	返回条数	扫描文档	扫描索引项	耗时
{status:"逾期"}	全表扫描（hint $natural）	429	3000	0	4 ms
{status:"逾期"}	走 idx_status	429	429	429	3 ms
{book_isbn:"9787115428028"}	全表扫描	60	3000	0	3 ms
{book_isbn:"9787115428028"}	走 idx_book_isbn	60	60	60	3 ms
分片侧 {book_isbn}	走 idx_book_isbn（跨 mongos）	60	60	60	40 ms
分片侧 {reader_card}	走 idx_card_hashed	100	100	100	3 ms

表 11 的读法是：全表扫描无论如何都要看 3000 个文档，索引扫描只看命中的那些（429 或 60），文档扫描量降了一个数量级。耗时在这批数据上差别不大（3-4 ms），因为 3000 条对内存来说太小；但扫描量是结构性的差异，数据涨到百万级时，4 ms 会变成秒级，而索引扫描仍是毫秒级。分片侧的 40 ms 高于单机的 3 ms，差额就是 mongos 转发与多分片归并的成本——这也是分片集群的固定开销，不是索引失效。

七、安全加固

实验环境也照生产的做法做了三件事：认证、最小权限、把系统层面的干扰项关掉。

表 12 安全措施与验证

层面	措施	验证方式
传输认证	所有实例开启 keyFile 内部认证，集群成员之间互信	未带 keyFile 的 mongod 无法加入副本集
用户认证	security.authorization: enabled，三层账号	不带账号连接报 Unauthorized
最小权限	自定义 library_editor 角色（find/insert/update/remove）	lib_editor 删集合返回 Unauthorized
系统层	firewalld disable、SELinux 改 permissive 并 setenforce 0	ss -lntp 可见实例监听 0.0.0.0
密钥管理	mongo.key 三台一致，权限 400，仅 root 可读	ls -l /data/lib/key/mongo.key

这一条要写清楚：把 SELinux 改成 permissive、把 firewalld 关掉，是为了排除“连不上到底是认证问题还是防火墙问题”这类干扰，属于实验取舍，不是生产建议。真实部署应当保留 enforcing 并为 27017-27022 精确放行，同时把 bindIp 收窄到集群内部网段，而不是 0.0.0.0。

八、典型故障与排查记录

这一章记的都是真的卡住过、后来又照着报错找到原因的问题，日志/ 目录里能对上。

表 13 典型故障与处理

现象	根因	处理
shardCollection 报 InvalidOptions	哈希分片键缺少前缀索引	先建 {isbn:"hashed"} / {reader_card:"hashed"} 索引，再分片
split 报 NamespaceNotSharded	mongoimport --drop 把分片元数据一起清掉	调整顺序为 导入 → 建索引 → 分片
WiredTiger WT_PANIC，exitCode 48	进程未退出就删数据目录	先按端口找到 pid 并 kill -9，再清目录
清理旧实例时终端断开，命令 exit -1	pkill -f 的模式匹配到执行命令的进程自身	改成 ss -lntp 解析出 pid，再 kill -9
createUser 报 requires authentication	mongosh 默认连 ::1，不享受 localhost 例外	改用 127.0.0.1 + directConnection=true
GridFS get 报 not found	mongofiles 把绝对路径当文件名	cd 进目录，用相对文件名上传
GridFS 上传全部 Unauthorized	开启认证后未带账号	加 -u/-p 与 --authenticationDatabase
$round 报 unary operator	$round 不能当 $group 累加器	先 $avg，再在 $project 里 $round
mongod 前台启动占住终端	启动命令没脱离当前会话	setsid 后台化、输出重定向到 /dev/null，改轮询端口
副本集脚本重复执行报 AlreadyInitialized	没判断当前实例是否已在副本集里	先看 db.hello().setName，已是 rs_lib 就只打印状态

这十条里有六条是“顺序/前置条件”类错误，而不是语法错误：分片要先建索引，导入会破坏分片元数据，删目录要先停进程。数据库运维里这类错误的代价往往比写错一条命令大得多，值得单独列出来。

九、网络模式切换与遗留实例清理

9.1 桥接模式的尝试

按老师给的思路，第一选择是桥接模式，让三台虚拟机像局域网内的独立主机一样被访问。实施步骤是：备份三台 vmx 文件 → 把 ethernet0.connectionType 从 nat 改为 bridged → 启动 → 客户机改 DHCP 或配静态 IP。

表 14 桥接模式尝试记录

步骤	预期	实际结果
改 vmx 为 bridged 并启动	虚拟机拿到本机 LAN 网段（192.168.1.x）的地址	三台仍显示 192.168.121.x（NAT 侧地址），ARP 表里 MAC 未变
客户机改 DHCP	从家用路由器分到 192.168.1.x	没有拿到任何 IPv4，只剩 fe80 链路本地地址
手动配静态 192.168.1.201/24，网关 192.168.1.1	宿主机 ping 通、SSH 通	客户机 ip -br addr 显示配置成功，但宿主机 ping 与 22 端口探测均为 False
检查 VMnet0 绑定	VMnet0 桥到 WLAN 网卡	C:\ProgramData\VMware\vmnetbridge.cfg 不存在，注册表里也没有 VMnet0/Bridge 项，无法确认桥接对象

结论是桥接通道在本机没有真正建立：本机只有一块 Intel Wi-Fi 6E AX211 无线网卡在用（192.168.1.6/24，网关 192.168.1.1），另外还有一个 Radmin VPN 虚拟网卡；VMware 的桥接服务

没有把 VMnet0 绑到无线网卡上，无线网卡本身对“混杂模式/多 MAC”的支持也因驱动与路由器策略而异。这三者叠加，导致虚拟机发出去的帧出不了宿主。

9.2 回退 NAT

做法是：客户机用 nmcli 把 ens33 改回原静态（.134/.135/.136，网关 192.168.121.2，DNS 223.5.5.5）→ 关机 → 用备份的 vmx（NOSQL01.vmx.nat.bak 等）覆盖回去 → 启动 → 复测 22 端口。三台 port22 全部为 True，实例重新拉起后功能完好。改之前的 vmx 文件和网卡配置都在本机留了备份，需要再次尝试桥接时可以原样回退。

本次失败不影响项目验收：任务书要求的是“三台虚拟机 + MongoDB 集群”，NAT 模式下三台互通、宿主机可 SSH、集群功能与网络模式无关。桥接失败的真实原因是宿主网卡与 VMware 桥接服务的绑定问题，属于环境限制，不是配置写错。若将来要重试，正确的下一步是先在 VMware 虚拟网络编辑器里确认 VMnet0 已桥到 WLAN，而不是继续在客户机里改 IP。

9.3 遗留实例清理

三台上原本都带一份上一门课遗留的 mongod（/etc/mongod.conf，端口 27016，数据 204 MB）。按“直接删掉”的要求执行：systemctl disable --now mongod → 备份配置到 /data/lib/tmp/legacy_mongod.conf.bak → 删 /etc/mongod.conf → 删 /var/lib/mongo（204/205/204 MB）→ 确认 27016 已释放。清理后各机端口只剩本项目使用的那些，不再有第二个 MongoDB 抢端口或抢缓存的隐患。

十、验收结论

最后一次完整验收（12_集群验收.sh，集群重启后又跑了一遍）覆盖 6 类 23 个检查点，全部通过。

表 15 最终验收结果

检查类别	检查项	结果
节点	三台主机名、IP、系统、内核、内存	nosql01/02/03 分别为 .134/.135/.136；CentOS Stream 10；kernel 6.12.0-269；内存 3 GB 可用
端口	每台实例监听情况	nosql01：27017/27018/27019/27020/27022；nosql02：27018/27019/27020/27021；nosql03：27018/27019/27021
单机	数据量、GridFS、索引数	50 / 30 / 3000，fs.files 6、fs.chunks 6，索引 5 / 4 / 6
副本集	成员状态、复制延迟	01 PRIMARY，02/03 SECONDARY，health 全 1，replLag 0
分片	分片挂载、块与文档分布	shard1/shard2 均在线；borrow_logs 1300/1700，books 21/29
路由	带/不带分片键的查询落点	带键 → shard1；不带键 → shard1,shard2
性能	索引扫描与全表扫描对比	扫描文档 3000 → 429 / 60

总的来说：单机、副本集、分片集群这三套东西在同一组三台虚拟机上都跑通了，覆盖了任务书要求的检索、统计、索引、权限、文件存储、故障转移、分片与路由七类能力；所有数字均来自机器上的实际输出，可按附录 A 的命令逐条复现。做得还不够的地方：截图大多是虚拟机控制台和终端输出，Xshell 窗口的截图只有一张；分片集群的两个分片共用了 nosql02 一台机器，冗余度低于生产标准；桥接模式未能成功，最终以 NAT 交付。

附录 A 关键命令复现清单

```
# 1 导入数据（三个集合，认证未开时不用带账号）
bash 脚本/01_导入数据.sh 27017

# 2 单机数据量
```

```
mongosh --port 27017 -u platform_admin -p Admin2026pwd --authenticationDatabase admin \
  --eval 'const d=db.getSiblingDB("library"); print(d.books.countDocuments(),
d.readers.countDocuments(), d.borrow_logs.countDocuments())'

# 3 副本集状态
mongosh --port 27018 -u platform_admin -p Admin2026pwd --authenticationDatabase admin \
  --eval 'rs.status().members.forEach(m=>print(m.name, m.stateStr))'

# 4 分片与块分布
mongosh --port 27022 -u platform_admin -p Admin2026pwd --authenticationDatabase admin \
  --eval 'const d=db.getSiblingDB("library");
printjson(d.borrow_logs.getShardDistribution().value)'

# 5 路由验证
mongosh --port 27022 -u platform_admin -p Admin2026pwd --authenticationDatabase admin \
  --eval
'printjson(db.getSiblingDB("library").borrow_logs.find({reader_card:"R20240001"}).explain().query
Planner.winningPlan.shards)'

# 6 全量验收
bash 脚本/12_集群验收.sh
```

附录 B 截图与日志清单

表 16 截图与日志

文件	内容
截图/VMware 三台运行.png	VMware Workstation 三台虚拟机开机状态
截图/Xshell 连接 nosql01.png	Xshell 登录 nosql01 的 SSH 会话
截图/单机数据与索引.png	单机数据量、GridFS、索引数量
截图/副本集三节点.png	副本集成员状态与复制延迟
截图/虚拟机 01~03 控制台.png	三台虚拟机控制台画面
日志/20_集群验收.log	验收那一段的完整命令与输出
日志/	各阶段部署与排错日志（22 份阶段日志、5 份网络排查记录）